Table des matières

Exercices corrigés 09 — Séries temporelles et as-of joins 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 5
Mises en situation 6
Corrigé 7

Exercices corrigés 09 — Séries temporelles et as-of joins

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL, tantôt sur le transactionnel, tantôt sur l'entrepôt ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez quatre heures et trois quarts pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué `[OLTP]` se traite dans PostgreSQL sur `Adventureworks` (schémas `sales`, `production`, `person`). Un énoncé marqué `[OLAP]` se traite dans DuckDB sur `adventureworksdw.duckdb`, ouvert par `USE dbo;`. Sans marque, la question vaut pour les deux.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Distinguez en une phrase chacune un instant, une période et un intervalle de validité.
2. Quelle erreur produit la confusion entre un instant et une période ? Donnez la forme d'écriture qui l'évite.
3. Pourquoi `BETWEEN '2023-01-01' AND '2023-12-31'` est-il dangereux sur une colonne horodatée, et pourquoi le danger peut-il rester invisible pendant des années ?
4. Une série issue d'un `GROUP BY` sur le mois ne montre pas les mois vides. Expliquez pourquoi, et dites comment on les fait apparaître.
5. Sur une série trouée, la moyenne calculée sur l'axe complet et la moyenne calculée sur les jours actifs diffèrent. Laquelle est fausse ?

© 2026 iAlexMG. Tous droits réservés.

6. À quoi sert `NULLIF` dans le calcul d'une variation relative, et que se passe-t-il si on l'oublie ?
7. Une hausse de 50 % suivie d'une baisse de 50 % laisse à combien de la valeur initiale ? Que démontre ce résultat ?
8. Donnez la propriété du rendement logarithmique qui le rend préférable pour cumuler, et le prix qu'on paie pour cette propriété.
9. `ROWS BETWEEN 6 PRECEDING` et `RANGE BETWEEN INTERVAL '6 days' PRECEDING`. Sur quelle sorte de série donnent-ils le même résultat, et sur quelle sorte divergent-ils ?
10. Vous voulez la moyenne des sept derniers jours d'un indicateur de tendance. Quel cadre choisissez- vous, et quelle est la parade portable si le moteur ne le supporte pas ?
11. Quel cadre le moteur applique-t-il quand un `OVER` porte un `ORDER BY` sans cadre explicite ? Pourquoi ce défaut est-il gênant sur une série temporelle ?
12. `DATE_TRUNC('week', d)` ne donne pas forcément le même résultat sur les deux moteurs. Expliquez.
13. Pourquoi `LAST_VALUE` sans cadre explicite renvoie-t-il la ligne courante ? Donnez le cadre à écrire.
14. Définissez un as-of join en une phrase, sans employer de nom de fonction.
15. Écrivez de mémoire la forme portable de l'as-of join, et dites à quoi sert le mot `LEFT`.
16. Pourquoi l'as-of join s'appuie-t-il sur la date de début et jamais sur l'intervalle fermé ? Donnez les deux défauts de données auxquels cette règle rend insensible.

1. Dans la sous-requête d'un as-of join, la comparaison s'écrit `<=` plutôt que `<`. Qu'est-ce que cela change concrètement ?
2. Décrivez le motif de résolution des îlots et lacunes, en deux étapes. Pourquoi la somme cumulée donne-t-elle un identifiant de session ?
3. `PERCENTILE_CONT` et `PERCENTILE_DISC` renvoient parfois deux nombres différents. Expliquez la différence, et donnez un cas où l'interpolation est illégitime.
4. Une cohorte de mars n'a pas encore atteint son rang 6. Quelle valeur de rétention faut-il afficher à ce rang, et pourquoi celle-là ?
5. Combien d'heures compte la journée du 1er novembre 2026 à Montréal, et que rend la soustraction de ses deux minuits sur chacun de nos deux moteurs ?
6. `TIME_BUCKET` et `DATE_BIN`. Sur quel moteur vit chacune, et pourquoi deux quinzaines de même largeur peuvent-elles ne pas avoir les mêmes bornes d'un moteur à l'autre ?

Questions pratiques

© 2026 iAlexMG. Tous droits réservés.

P1. [OLTP] Construisez la série quotidienne du nombre de commandes sur l'année 2023, avec un axe complet et un zéro sur les jours sans commande. Renvoyez aussi une colonne qui distingue un zéro observé d'un zéro reconstitué.

P2. [OLAP] Écrivez la requête qui donne, par mois, le total des ventes, la variation en valeur, la variation en pourcentage et le rendement logarithmique. Protégez la division.

P3. [OLTP] Les achats cette fois : construisez la série quotidienne du montant des bons de commande de `purchasing.purchaseorderheader`, la somme de `subtotal` par jour de `orderdate`. Calculez la moyenne mobile à sept jours en cadre de lignes et en cadre de durée, sur la série brute, puis sur l'axe complet. Dites laquelle des colonnes obtenues répond à « le montant moyen acheté par jour sur les sept derniers jours », et ce que calculent les autres.

P4. [OLTP] Chaque représentant reçoit un quota de ventes, révisé par trimestre dans `sales.salespersonquotahistory`. Écrivez l'as-of join qui rattache à chaque commande passée par un représentant le quota en vigueur à sa date. Comptez les commandes sans quota, puis refaites le compte en remplaçant `<=` par `<`, et dites ce que ce second compte ne montre pas.

P5. [OLAP] Retournez l'as-of join. Pour chaque nouvelle version d'un produit de `dbo.DimProduct`, c'est-à-dire chaque version qui a une version antérieure de même `ProductAlternateKey`, trouvez la première vente du produit à partir de son entrée en vigueur, et le délai qui les sépare. Écrivez-le une fois en `LATERAL` et une fois en `QUALIFY`, et vérifiez que les deux donnent les mêmes comptes.

P6. [OLTP] Écrivez les trois requêtes de contrôle de l'historique des affectations des employés à leurs services, `humanresources.employeedepartmenthistory` : intervalles incohérents, chevauchements, versions courantes multiples. Regardez ensuite les employés qui ont changé de service, et dites comment une date de fin s'enchaîne avec la date de début suivante, et ce que cela impose à un as-of join sur cette table.

P7. [OLTP] Découpez les commandes du client 29825 en sessions séparées par plus de 120 jours d'inactivité, puis renvoyez pour chaque session sa date de début, sa date de fin, le nombre de commandes et le montant total. (avancé)

P8. [OLAP] Construisez la table de rétention par cohorte mensuelle, du rang 0 au rang 6, en pourcentage de la taille de la cohorte. Ajoutez une colonne qui marque les cellules dont la valeur n'est pas encore observable. (avancé)

P9. [OLAP] La semaine, deux fois. Montrez sur `FactInternetSales` que `TIME_BUCKET` sur sept jours découpe exactement comme `DATE_TRUNC('week', …)`, puis qu'il cesse de le faire dès qu'on lui donne le 1er janvier 2000 pour origine. Expliquez les deux résultats.

P10. [OLTP] Les journées de 2026. En une requête, comptez les journées de 2026 qui ne durent pas 24 heures à Montréal, en mesurant chaque journée en secondes. (avancé)

Diagnostics

E1. Un rapport annuel sur PostgreSQL :

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT COUNT(*) FROM sales.salesorderheader
WHERE  orderdate BETWEEN '2023-01-01' AND '2023-12-31';
```

L'auteur affirme compter toutes les commandes de 2023. Dites dans quelles conditions il a raison, dans quelles conditions il a tort, et donnez l'écriture qui n'a jamais tort.

E2. Une moyenne mobile sur l'entrepôt :

```sql
SELECT jour, ventes,
       AVG(ventes) OVER (ORDER BY jour) AS moyenne_mobile_7j
FROM   ventes_jour;
```

L'auteur croit avoir écrit une moyenne mobile à sept jours. Dites ce qu'il a réellement calculé.

E3. Un rattachement de prix :

```sql
SELECT h.salesorderid, d.productid, p.listprice
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
JOIN   production.product AS p ON p.productid = d.productid;
```

La requête s'exécute, ne perd aucune ligne et renvoie un prix pour chaque commande. Elle est pourtant fausse pour l'usage prévu, qui est un rapport de marge historique. Expliquez.

E4. Un as-of join sur l'entrepôt :

```sql
SELECT f.SalesOrderNumber, v.ListPrice
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS ref ON ref.ProductKey = f.ProductKey
JOIN   dbo.DimProduct AS v
       ON v.ProductAlternateKey = ref.ProductAlternateKey
      AND f.OrderDate BETWEEN v.StartDate AND v.EndDate;
```

Elle renvoie zéro ligne. Donnez les deux causes, et l'écriture correcte.

E5. Un as-of join sur PostgreSQL :

```sql
SELECT h.salesorderid, prix.listprice
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
JOIN   LATERAL (
         SELECT pl.listprice
         FROM   production.productlistpricehistory AS pl
         WHERE  pl.productid = d.productid
           AND  pl.startdate <= h.orderdate
```


© 2026 iAlexMG. Tous droits réservés.

```
          ORDER  BY pl.modifieddate DESC
          LIMIT  1
        ) AS prix ON true;
```

Deux défauts, dont un seul est visible sur cette base. Nommez-les tous les deux. (avancé)

E6. Une sessionisation :

```
WITH marque AS (
    SELECT customerid, orderdate,
           CASE WHEN orderdate - LAG(orderdate) OVER (ORDER BY orderdate) > INTERVAL '120 days'
                THEN 1 ELSE 0 END AS rupture
    FROM   sales.salesorderheader
)
SELECT customerid, orderdate,
       SUM(rupture) OVER (PARTITION BY customerid ORDER BY orderdate) AS session
FROM   marque;
```

Le résultat est incohérent : des clients n'ont qu'un seul numéro de session alors qu'ils ont commandé sur trois ans, et d'autres en reçoivent un qui ne doit rien à leurs propres commandes. Trouvez l'erreur, puis dites ce qu'il manque encore à cette requête pour rendre deux fois le même résultat.

E7. Un tableau de bord d'énergie calcule la puissance moyenne de chaque journée à partir d'une série horaire d'instants :

```
SELECT CAST(instant AT TIME ZONE 'America/Toronto' AS DATE) AS jour,
       SUM(kwh) / 24                                        AS kw_moyens
FROM   releves_horaires
GROUP  BY 1;
```

Deux fois par an, la courbe fait un saut que personne n'explique. Dites quelles journées, dans quel sens, et corrigez.

Questions à développer

A1. La théorie affirme que l'as-of join doit s'appuyer sur la date de début et jamais sur l'intervalle fermé. Défendez cette règle en un paragraphe, puis dites ce qu'elle coûte : nommez au moins une situation où cette écriture donne un résultat que l'intervalle fermé aurait refusé, et dites si ce comportement est souhaitable.

A2. Le laboratoire OLAP montre que l'as-of join recalculé contredit la clé du fait sur 6 899 lignes. Trois lectures sont possibles. Exposez-les, dites laquelle vous privilégiez et par quelle mesure vous trancheriez. Précisez ce que vous feriez si vous ne pouviez pas trancher.

A3. Comparez la façon dont le monde transactionnel et l'entrepôt conservent l'historique d'un attribut. Votre réponse doit expliquer pourquoi l'un place l'historique dans une table dédiée et l'autre le replie dans la dimension, et dire où se logent les erreurs dans chaque cas.

© 2026 iAlexMG. Tous droits réservés.

A4. Les trois derniers modules ont chacun montré une faute qui ne lève pas d'erreur : le `NOT IN` sur colonne nullable, la valeur `JSON` comparée sans la flèche double, et l'intervalle de validité non contrôlé. Dégagez ce que ces trois fautes ont en commun, et proposez une pratique d'équipe qui les attraperait toutes les trois avant la production. (avancé)

Mises en situation

M1. Gestion de portefeuille Havre-Blanc, la valorisation à la date de l'ordre. Havre-Blanc tient deux tables. La première enregistre les ordres exécutés : un identifiant, un horodatage à la seconde, un symbole, un sens, une quantité. La seconde enregistre les cours : un symbole, un horodatage, un prix. Les cours n'arrivent pas à intervalle fixe, et il n'y a jamais de cours à la seconde exacte d'un ordre. La conformité demande, pour chaque ordre, le dernier cours coté avant l'exécution, plus l'écart entre le prix d'exécution et ce cours.

Décrivez la requête que vous écririez, en nommant l'outil et en justifiant chaque clause. Dites ce que vous faites des ordres pour lesquels aucun cours antérieur n'existe, et pourquoi ce cas doit rester visible. Précisez enfin ce qui changerait si la conformité demandait le cours suivant plutôt que le précédent.

M2. Coopérative de crédit Kamouraska, la courbe de taux. La coopérative affiche chaque jour une courbe de taux : pour cinq échéances, le taux en vigueur. Les taux ne changent pas tous en même temps ; chaque échéance a son propre historique de révisions, avec une date d'entrée en vigueur. L'équipe veut un tableau qui donne, pour chaque jour ouvrable des trois dernières années et pour chacune des cinq échéances, le taux applicable ce jour-là, même les jours où aucune révision n'a eu lieu.

Décrivez votre construction en trois étapes, en nommant l'outil de chacune. Dites pourquoi la question ne peut pas se résoudre par une jointure d'égalité, et ce qui se passerait si vous omettiez la première étape. Précisez comment vous traitez le début de la plage, avant la première révision connue.

M3. Parc éolien Anse-Verte, la volatilité glissante. Le parc reçoit une mesure de production toutes les dix minutes, par turbine, avec des interruptions de télémétrie qui durent de quelques minutes à plusieurs heures. L'exploitant veut, par turbine et par jour, l'écart type de la production sur une fenêtre glissante de vingt-quatre heures, et il veut pouvoir comparer les turbines entre elles.

Cette demande contient un piège que l'énoncé ne nomme pas. Trouvez-le, dites ce qu'il fait au résultat, et proposez la construction qui le désamorce. Votre réponse doit trancher entre le cadre en lignes et le cadre en durée, et dire ce que vous faites des périodes d'interruption : une valeur de zéro, une valeur manquante, ou une exclusion de la fenêtre. Défendez le choix, et dites quelle information vous ajouteriez au tableau pour que l'exploitant sache quand ne pas lire le chiffre. (avancé)

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés.

Questions courtes

1. Un instant est un point sur l'axe, sans durée. Une période est un seau qui a un début et une fin et qui pave l'axe sans trou ni recouvrement. Un intervalle de validité est un état vrai depuis une date, jusqu'à une autre ou jusqu'à nouvel ordre.

Théorie : `Theorie_09_SQL.md`, section 1.

2. Le décalage d'un jour : on compare un instant à une borne de période et on ampute la dernière journée. L'écriture qui l'évite est l'intervalle semi-ouvert, `>= début AND < fin`.

Théorie : `Theorie_09_SQL.md`, section 1.

3. Parce que `'2023-12-31'` se lit comme `2023-12-31 00:00:00` et que tout ce qui est arrivé après minuit ce jour-là tombe hors du filtre. Le danger reste invisible tant que la colonne ne porte que des minuits, ce qui est le cas de ce portage : la requête renvoie alors le bon nombre. Elle deviendra fausse le jour où l'application enregistrera l'heure réelle, et elle continuera de renvoyer un nombre plausible.

Théorie : `Theorie_09_SQL.md`, section 1.

4. Parce qu'un `GROUP BY` ne peut grouper que des lignes qui existent : un mois sans vente n'a aucune ligne à grouper. On les fait apparaître en générant l'axe des périodes, puis en y raccrochant les mesures par un `LEFT JOIN`.

Théorie : `Theorie_09_SQL.md`, section 2.

5. Aucune. Elles répondent à deux questions distinctes, la moyenne de la période et la moyenne des jours actifs. Ce qui serait faux, c'est de publier l'une en croyant avoir calculé l'autre.

Théorie : `Theorie_09_SQL.md`, section 2.

6. À transformer un dénominateur nul en `NULL`, ce qui rend le résultat `NULL` au lieu de faire tomber la requête sur une division par zéro. Sans lui, une série complétée par des zéros produit l'erreur dès le premier jour actif qui suit un jour vide.

Théorie : `Theorie_09_SQL.md`, section 3.

7. À 75 % de la valeur initiale. Cela démontre que les rendements simples ne s'additionnent pas, parce que les deux pourcentages ne portent pas sur la même base.

Théorie : `Theorie_09_SQL.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

8. Le rendement logarithmique d'une période longue est la somme des rendements logarithmiques des périodes courtes qui la composent, ce qui permet de cumuler, de moyenner et de calculer un écart type qui veut dire quelque chose. Le prix est le domaine de définition : le logarithme refuse zéro et refuse le négatif, donc il faut filtrer.

Théorie : `Theorie_09_SQL.md`, section 3.

9. Ils donnent le même résultat sur une série complète, une ligne par jour. Ils divergent dès que la série est trouée ou qu'une même date porte plusieurs lignes.

Théorie : `Theorie_09_SQL.md`, section 4.

10. Le cadre en durée, parce que la question porte sur une durée et non sur un nombre d'observations. La parade portable consiste à compléter l'axe des dates, puis à écrire `ROWS` : sur une série sans trou, les deux cadres décrivent le même ensemble.

Théorie : `Theorie_09_SQL.md`, section 4.

11. `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. Il est gênant sur une série temporelle parce que `RANGE` regroupe les valeurs d'ordre égales : si plusieurs lignes portent la même date, elles sont toutes ajoutées d'un coup, et un cumul censé avancer ligne par ligne avance par paquets.

Théorie : `Theorie_09_SQL.md`, section 4.

12. Parce que le premier jour de la semaine n'est pas le même partout. Nos deux terrains sont d'accord et suivent ISO 8601, en faisant commencer la semaine le lundi : `DATE_TRUNC('week', …)` appliqué au dimanche 17 mars 2024 rend le lundi 11 des deux côtés. D'autres moteurs laissent un paramètre de session en décider, `WEEK_START` sur Snowflake d'après sa documentation, et un rapport hebdomadaire peut alors changer de valeur sans qu'une ligne de SQL ait bougé.

Théorie : `Theorie_09_SQL.md`, section 5.

13. Parce que le cadre par défaut s'arrête à la ligne courante : `LAST_VALUE` renvoie donc la dernière ligne du cadre, qui est la ligne courante. Il faut écrire `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

Théorie : `Theorie_09_SQL.md`, section 4.

14. Une jointure qui rattache à chaque événement la dernière valeur connue au moment où il s'est produit, alors qu'aucune égalité de date n'existe entre les deux séries.

Théorie : `Theorie_09_SQL.md`, section 6.

15. La forme est `LEFT JOIN LATERAL (SELECT … WHERE cle = … AND debut <= evenement ORDER BY debut DESC LIMIT 1) AS x ON true`. Le `LEFT` garde les lignes pour lesquelles aucune version antérieure n'existe : sans lui, elles disparaissent du résultat sans que rien ne le signale.

Théorie : `Theorie_09_SQL.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

16. Parce que la date de début suffit à désigner la version en vigueur : c'est la plus récente qui a commencé avant l'événement. Cette écriture rend insensible à une date de fin absente, qui casse toute comparaison par `NULL`, et à une date de fin incohérente, qui décrit un intervalle vide.

Théorie : `Theorie_09_SQL.md`, section 6.

17. Elle décide du sort des événements qui tombent le jour même de l'entrée en vigueur. Avec `<=`, la nouvelle valeur s'applique ce jour-là ; avec `<`, c'est l'ancienne, ou rien s'il s'agit de la première version. Sur `Adventureworks`, le passage de `<=` à `<` laisse 2 898 lignes de commande sans prix, et en fait passer 1 191 autres à l'ancien prix, sans le moindre `NULL` pour le signaler.

Théorie : `Theorie_09_SQL.md`, section 6.

18. Un drapeau qui vaut 1 quand l'écart avec l'événement précédent dépasse le seuil, puis une somme cumulée de ce drapeau. La somme cumulée donne un identifiant de session parce qu'elle n'augmente qu'aux ruptures : toutes les lignes situées entre deux ruptures partagent donc la même valeur.

Théorie : `Theorie_09_SQL.md`, section 7.

19. `PERCENTILE_DISC` renvoie une valeur qui existe dans les données, `PERCENTILE_CONT` interpole entre les deux valeurs qui encadrent le point de coupe et peut donc renvoyer un nombre jamais observé. L'interpolation est illégitime sur une grandeur discrète ou catégorielle : la médiane interpolée d'un nombre d'enfants par ménage vaut 1,5, ce qui ne décrit aucun ménage.

Théorie : `Theorie_09_SQL.md`, section 8.

20. Aucune : la case doit rester vide, ou porter une marque explicite. La cohorte n'a pas eu six mois pour revenir, donc sa rétention à ce rang est inconnue et non nulle. Afficher zéro serait un mensonge par présentation, et il ferait plonger la courbe exactement là où le lecteur regarde en premier.

Théorie : `Theorie_09_SQL.md`, section 9.

21. Vingt-cinq heures : à 2 h, l'horloge revient à 1 h, et l'heure de 1 h à 2 h passe deux fois. La soustraction des deux minuits rend `1 day 01:00:00` sur PostgreSQL, et `1 day` sur DuckDB, qui compte en jours de calendrier ; `DATE_DIFF('hour', …)` y rend 25. La durée qui ne trompe pas se calcule en secondes, par `EXTRACT(EPOCH FROM …)`, sur les deux.

Théorie : `Theorie_09_SQL.md`, section 1.

22. `TIME_BUCKET` vit sur DuckDB, `DATE_BIN` sur PostgreSQL, et chacun ignore l'autre. Une période de largeur libre se compte à partir d'une origine : `TIME_BUCKET` en prend une d'office, le lundi 3 janvier 2000, alors que `DATE_BIN` exige qu'on la donne. Une requête portée avec une autre origine, le 1er janvier 2000 par exemple, garde la même largeur et décale toutes les bornes de deux jours.

Théorie : `Theorie_09_SQL.md`, section 5.

Questions pratiques

© 2026 iAlexMG. Tous droits réservés.

P1. [OLTP]

```sql
WITH axe AS (
    SELECT g::date AS jour
    FROM   generate_series(DATE '2023-01-01', DATE '2023-12-31', INTERVAL '1 day') AS g
),
brut AS (
    SELECT orderdate::date AS jour, COUNT(*) AS commandes
    FROM   sales.salesorderheader
    WHERE  orderdate >= '2023-01-01' AND orderdate < '2024-01-01'
    GROUP  BY 1
)
SELECT a.jour,
       COALESCE(b.commandes, 0) AS commandes,
       (b.jour IS NULL)         AS jour_reconstitue
FROM   axe AS a
LEFT   JOIN brut AS b ON b.jour = a.jour
ORDER  BY a.jour;
```

La colonne `jour_reconstitue` est le point de l'exercice : un zéro observé et un zéro inventé n'ont pas la même valeur de preuve, et la trace de ce qu'on a fabriqué doit survivre à la requête.

Théorie : `Theorie_09_SQL.md`, section 2.

P2. [OLAP]

```sql
WITH mensuel AS (
    SELECT DATE_TRUNC('month', OrderDate) AS mois, SUM(SalesAmount) AS ventes
    FROM   dbo.FactInternetSales
    GROUP  BY 1
)
SELECT mois,
       ventes,
       ventes - LAG(ventes) OVER (ORDER BY mois) AS variation,
       ROUND(100 * (ventes - LAG(ventes) OVER (ORDER BY mois))
             / NULLIF(LAG(ventes) OVER (ORDER BY mois), 0), 1) AS variation_pct,
       LN(ventes / NULLIF(LAG(ventes) OVER (ORDER BY mois), 0)) AS rendement_log
FROM   mensuel
ORDER  BY mois;
```

Est également acceptable une réponse qui calcule le `LAG` une seule fois dans une CTE et le réutilise trois fois : c'est plus lisible, et le moteur ne s'en portera pas plus mal.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_09_SQL.md`, section 3.

P3. [OLTP]

```sql
-- La série brute : seulement les jours où l'on a acheté.
WITH achats AS (
    SELECT orderdate::date AS jour, SUM(subtotal) AS montant
    FROM   purchasing.purchaseorderheader
    GROUP  BY 1
)
SELECT jour, montant,
       AVG(montant) OVER (ORDER BY jour ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)   AS mm7_lignes,
       AVG(montant) OVER (ORDER BY jour RANGE BETWEEN INTERVAL '6 days' PRECEDING
                                          AND CURRENT ROW)                          AS mm7_duree
FROM   achats
ORDER  BY jour;

-- L'axe complet : un jour de calendrier par ligne, zéro les jours sans achat.
WITH achats AS (
    SELECT orderdate::date AS jour, SUM(subtotal) AS montant
    FROM   purchasing.purchaseorderheader
    GROUP  BY 1
),
axe AS (
    SELECT d::date AS jour
    FROM   generate_series((SELECT MIN(jour) FROM achats), (SELECT MAX(jour) FROM achats),
                           INTERVAL '1 day') AS d
)
SELECT a.jour,
       COALESCE(x.montant, 0) AS montant,
       AVG(COALESCE(x.montant, 0)) OVER (ORDER BY a.jour
                                         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS mm7_calendrier
FROM   axe AS a
LEFT   JOIN achats AS x ON x.jour = a.jour
ORDER  BY a.jour;
```

On n'achète que 300 jours sur les 1 256 de la période, avec des trous qui vont jusqu'à 228 jours. Sur la série brute, les deux cadres divergent sur 284 lignes sur 300, et le 11 juin 2023 montre les trois réponses d'un coup. Le cadre en lignes remonte sept achats, jusqu'au 9 mars, et rend 133 616,09 $; le cadre en durée ne trouve que ce jour-là dans ses sept jours, et rend 27 381,47 $, le montant du jour ; l'axe complet rend 3 911,64 $, ce montant divisé par sept. Sur l'axe complet, les deux cadres coïncident sur les 1 256 lignes, mesuré.

© 2026 iAlexMG. Tous droits réservés.

Seule la colonne de l'axe complet répond à la question posée. Le cadre en lignes calcule la moyenne des sept derniers achats, qui peuvent couvrir des mois ; le cadre en durée, sur la série brute, la moyenne par jour d'achat des sept derniers jours, parce qu'un jour absent ne compte pas au dénominateur. Les deux sont des questions légitimes, et aucune n'est celle de l'énoncé.

Théorie : `Theorie_09_SQL.md`, section 4.

P4. `[OLTP]`

```sql
SELECT COUNT(*) AS commandes,
       COUNT(*) FILTER (WHERE q_le.salesquota IS NULL)           AS sans_quota_le,
       COUNT(*) FILTER (WHERE q_lt.salesquota IS NULL)           AS sans_quota_lt,
       COUNT(*) FILTER (WHERE q_lt.salesquota <> q_le.salesquota) AS quota_change
FROM   sales.salesorderheader AS h
LEFT   JOIN LATERAL (
         SELECT sq.salesquota
         FROM   sales.salespersonquotahistory AS sq
         WHERE  sq.businessentityid = h.salespersonid
           AND  sq.quotadate <= h.orderdate
         ORDER  BY sq.quotadate DESC
         LIMIT  1
       ) AS q_le ON true
LEFT   JOIN LATERAL (
         SELECT sq.salesquota
         FROM   sales.salespersonquotahistory AS sq
         WHERE  sq.businessentityid = h.salespersonid
           AND  sq.quotadate < h.orderdate
         ORDER  BY sq.quotadate DESC
         LIMIT  1
       ) AS q_lt ON true
WHERE  h.salespersonid IS NOT NULL;
```

Attendu : 3 806 commandes, toutes avec un quota. Avec `<`, 67 commandes perdent le leur : celles qu'un représentant a passées le jour même de son premier quota. Le second compte ne montre pas le reste : 748 autres commandes, passées le jour d'une révision, reçoivent sans bruit le quota du trimestre précédent, et seule la comparaison des deux colonnes les fait voir. C'est la leçon de la sous-partie 4.5 du labo, sur d'autres données : la faute qui coûte n'est pas celle qui laisse un `NULL`, c'est celle qui laisse une valeur plausible.

Théorie : `Theorie_09_SQL.md`, section 6.

P5. `[OLAP]`

```sql
-- En LATERAL.
SELECT COUNT(*) AS versions, COUNT(s.premiere) AS suivies_d_une_vente,
       MIN(s.premiere - v.StartDate) AS delai_min, MAX(s.premiere - v.StartDate) AS delai_max
```


© 2026 iAlexMG. Tous droits réservés.

```sql
FROM   dbo.DimProduct AS v
LEFT   JOIN LATERAL (
         SELECT f.OrderDate AS premiere
         FROM   dbo.FactInternetSales AS f
         JOIN   dbo.DimProduct AS p ON p.ProductKey = f.ProductKey
         WHERE  p.ProductAlternateKey = v.ProductAlternateKey
           AND  f.OrderDate >= v.StartDate
         ORDER  BY f.OrderDate
         LIMIT  1
       ) AS s ON TRUE
WHERE  EXISTS (SELECT 1 FROM dbo.DimProduct AS w
               WHERE w.ProductAlternateKey = v.ProductAlternateKey AND w.StartDate < v.StartDate);

-- En QUALIFY.
WITH s AS (
    SELECT v.ProductKey, v.StartDate, f.OrderDate AS premiere
    FROM   dbo.DimProduct AS v
    LEFT   JOIN dbo.DimProduct AS p ON p.ProductAlternateKey = v.ProductAlternateKey
    LEFT   JOIN dbo.FactInternetSales AS f ON f.ProductKey = p.ProductKey
                                          AND f.OrderDate >= v.StartDate
    WHERE  EXISTS (SELECT 1 FROM dbo.DimProduct AS w
                   WHERE w.ProductAlternateKey = v.ProductAlternateKey AND w.StartDate < v.StartDate)
    QUALIFY ROW_NUMBER() OVER (PARTITION BY v.ProductKey ORDER BY f.OrderDate NULLS LAST) = 1
)
SELECT COUNT(*) AS versions, COUNT(premiere) AS suivies_d_une_vente,
       MIN(premiere - StartDate) AS delai_min, MAX(premiere - StartDate) AS delai_max
FROM   s;
```

Les deux renvoient 102 nouvelles versions, dont 47 suivies d'une vente, de zéro à 190 jours après leur entrée en vigueur ; les 55 autres ne se sont jamais vendues ensuite. C'est l'as-of join à l'envers : le même `ORDER BY … LIMIT 1` ou le même `ROW_NUMBER`, mais trié vers l'avant. Le `LEFT` est indispensable dans les deux écritures, et le `NULLS LAST` dans la seconde : sans eux, les 55 versions jamais vendues disparaîtraient, et l'on croirait que toute nouvelle version trouve preneur.

Théorie : `Theorie_09_SQL.md`, sections 6 et 10.

P6. `[OLTP]`

```sql
-- 1. Intervalles incohérents.
SELECT COUNT(*) AS incoherents
FROM   humanresources.employeedepartmenthistory
WHERE  enddate IS NOT NULL AND enddate < startdate;

-- 2. Chevauchements pour un même employé.
SELECT COUNT(*) AS chevauchements
```


© 2026 iAlexMG. Tous droits réservés.

```sql
FROM   humanresources.employeedepartmenthistory AS a
JOIN   humanresources.employeedepartmenthistory AS b
       ON b.businessentityid = a.businessentityid
      AND b.startdate > a.startdate
      AND b.startdate <= COALESCE(a.enddate, DATE '9999-12-31');

-- 3. Plusieurs affectations ouvertes pour un même employé.
SELECT businessentityid, COUNT(*)
FROM   humanresources.employeedepartmenthistory
WHERE  enddate IS NULL
GROUP  BY businessentityid
HAVING COUNT(*) > 1;
```

Attendu : zéro, zéro, et aucune ligne. La table compte 296 affectations pour 290 employés, dont 290 ouvertes, une par employé. Un contrôle qui rend zéro est un résultat, pas une perte de temps : c'est lui qui autorise la jointure qui suit. Le `COALESCE` de la deuxième requête est ce qui la rend juste en général : sans lui, une version ouverte ne serait jamais vue comme chevauchant quoi que ce soit, alors qu'elle chevauche tout ce qui la suit.

Cinq employés ont changé de service, l'employé 250 deux fois. Chez tous, la date de fin est la veille de la date de début suivante : le numéro 4 quitte son premier service le 30 mai 2010 et entre dans le second le 31. Les fins sont donc inclusives. Un as-of join sur cette table s'écrit `jour BETWEEN startdate AND COALESCE(enddate, DATE '9999-12-31')`, bornes comprises, ou, mieux, sur la seule date de début, comme la section 6 le recommande ; une fin traitée comme exclusive perdrait le dernier jour de chaque affectation.

Théorie : `Theorie_09_SQL.md`, sections 1 et 6.

P7. `[OLTP]` (avancé)

```sql
WITH marque AS (
    SELECT customerid, salesorderid, orderdate::date AS jour, totaldue,
           CASE WHEN orderdate::date
                     - LAG(orderdate::date) OVER (PARTITION BY customerid
                                                  ORDER BY orderdate) > 120
                THEN 1 ELSE 0 END AS rupture
    FROM   sales.salesorderheader
    WHERE  customerid = 29825
),
sessions AS (
    SELECT customerid, salesorderid, jour, totaldue,
           SUM(rupture) OVER (PARTITION BY customerid ORDER BY jour
                              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS session
    FROM   marque
)
SELECT session,
```


© 2026 iAlexMG. Tous droits réservés.

```
       MIN(jour)          AS debut,
       MAX(jour)          AS fin,
       MAX(jour) - MIN(jour) AS duree_jours,
       COUNT(*)           AS commandes,
       SUM(totaldue)      AS montant
FROM   sessions
GROUP  BY session
ORDER  BY session;
```

Le filtre sur le client est placé dans la première CTE volontairement : il réduit le travail de la fenêtre au lieu de le jeter après coup. Le `PARTITION BY customerid` reste écrit malgré ce filtre, pour que la requête reste juste si l'on retire le `WHERE`.

Théorie : `Theorie_09_SQL.md`, section 7.

P8. `[OLAP]` (avancé)

```
WITH premiere AS (
    SELECT CustomerKey, DATE_TRUNC('month', MIN(OrderDate))::date AS mois_cohorte
    FROM   dbo.FactInternetSales GROUP BY CustomerKey
),
bornes AS (SELECT DATE_TRUNC('month', MAX(OrderDate))::date AS dernier_mois
           FROM dbo.FactInternetSales),
activite AS (
    SELECT f.CustomerKey, p.mois_cohorte, DATE_TRUNC('month', f.OrderDate)::date AS mois_actif
    FROM   dbo.FactInternetSales AS f
    JOIN   premiere AS p ON p.CustomerKey = f.CustomerKey
),
grille AS (
    SELECT mois_cohorte, DATEDIFF('month', mois_cohorte, mois_actif) AS rang,
           COUNT(DISTINCT CustomerKey) AS clients
    FROM   activite GROUP BY 1, 2
)
SELECT g.mois_cohorte, g.rang, g.clients,
       ROUND(100.0 * g.clients / MAX(CASE WHEN g.rang = 0 THEN g.clients END)
                                 OVER (PARTITION BY g.mois_cohorte), 1) AS retention_pct,
       (DATEDIFF('month', g.mois_cohorte, b.dernier_mois) < g.rang) AS non_observable
FROM   grille AS g CROSS JOIN bornes AS b
WHERE  g.rang <= 6
ORDER  BY g.mois_cohorte, g.rang;
```

La colonne `non_observable` est le point de l'exercice. Elle compare le rang demandé au nombre de mois réellement écoulés depuis la cohorte : au delà, la cellule ne dit rien, et le tableau doit le dire lui-même plutôt que de laisser le lecteur le deviner.

© 2026 iAlexMG. Tous droits réservés.

Est également acceptable une réponse qui génère la grille complète des rangs pour afficher un zéro sur les rangs sans activité, à condition de distinguer ce zéro du vide de la colonne ci-dessus. Les deux problèmes sont différents et méritent deux traitements.

Théorie : `Theorie_09_SQL.md`, section 9.

P9. `[OLAP]`

```
SELECT COUNT(*)                                                                    AS lignes,
       COUNT_IF(TIME_BUCKET(INTERVAL '7 days', OrderDate) <> DATE_TRUNC('week', OrderDate)) AS ecarts_par_defaut,
       COUNT_IF(TIME_BUCKET(INTERVAL '7 days', OrderDate, TIMESTAMP '2000-01-01')
                <> DATE_TRUNC('week', OrderDate))                                  AS ecarts_origine_1er
FROM   FactInternetSales;
```

60 398 lignes, 0 écart par défaut, 60 398 écarts avec l'origine du 1er janvier. L'origine implicite de `TIME_BUCKET`, le 3 janvier 2000, est un lundi, et une largeur de sept jours comptée depuis un lundi retombe sur les semaines ISO de `DATE_TRUNC`. Le 1er janvier 2000 est un samedi : chaque semaine commence alors un samedi, et aucune ne coïncide plus. La fonction n'a pas changé, seule l'origine l'a fait.

Théorie : `Theorie_09_SQL.md`, section 5.

P10. `[OLTP]` (avancé)

```
SELECT COUNT(*) AS journees_hors_24_heures
FROM   generate_series(DATE '2026-01-01', DATE '2026-12-31', INTERVAL '1 day') AS g(j)
WHERE  EXTRACT(EPOCH FROM ((j::date + 1)::timestamp AT TIME ZONE 'America/Toronto')
                        - (j::date::timestamp       AT TIME ZONE 'America/Toronto')) <> 86400;
```

Deux journées, le 8 mars et le 1er novembre. Chaque journée se mesure de son minuit au minuit suivant, tous deux lus comme des heures de mur de Montréal et donc changés en instants, et la différence se compte en secondes : 82 800 et 90 000, contre 86 400 pour une journée ordinaire. Le `::date` ramène l'horodatage de `generate_series` à une date avant d'y ajouter un jour.

Théorie : `Theorie_09_SQL.md`, section 1.

Diagnostics

E1. Il a raison si et seulement si la colonne ne porte que des minuits, ce qui est le cas de ce portage. Il a tort dès qu'une heure réelle apparaît : tout ce qui s'est passé le 31 décembre après minuit tombe hors du filtre, soit presque une journée entière par an. La gravité vient de ce que la requête continue de renvoyer un nombre plausible.

```
SELECT COUNT(*) FROM sales.salesorderheader
WHERE  orderdate >= '2023-01-01' AND orderdate < '2024-01-01';
```

Théorie : `Theorie_09_SQL.md`, section 1.

© 2026 iAlexMG. Tous droits réservés.

E2. Il a calculé un cumul depuis le début de la série, pas une moyenne mobile. En l'absence de cadre explicite, le moteur applique `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, donc chaque ligne moyenne toutes les valeurs depuis le premier jour. La courbe obtenue est lisse et croît doucement, ce qui ressemble beaucoup à une moyenne mobile et n'en est pas une.

```sql
SELECT jour, ventes,
       AVG(ventes) OVER (ORDER BY jour ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS mm7
FROM   ventes_jour;
```

Théorie : `Theorie_09_SQL.md`, section 4.

E3. Elle rattache le prix d'aujourd'hui, celui que porte `production.product`, à des commandes passées il y a trois ans. Rien ne manque et rien n'échoue, mais la marge historique calculée ainsi est un anachronisme : elle change chaque fois qu'un prix est modifié, donc le rapport de l'exercice clos ne donne pas deux fois le même chiffre. Il faut l'as-of join sur `production.productlistpricehistory`.

Théorie : `Theorie_09_SQL.md`, section 6.

E4. Deux causes. D'abord, les 200 dates de fin de `dbo.DimProduct` sont toutes antérieures à leur date de début, donc l'intervalle des versions historiques est vide et n'apparie personne. Ensuite, les 406 versions courantes ont `EndDate` à `NULL`, et une comparaison avec `NULL` ne vaut ni vrai ni faux, donc elles n'apparient personne non plus. Aucune ligne de la dimension ne peut apparier quoi que ce soit, et le résultat est vide sans erreur.

```sql
SELECT f.SalesOrderNumber, v.ListPrice
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS ref ON ref.ProductKey = f.ProductKey
LEFT   JOIN LATERAL (
         SELECT p.ListPrice
         FROM   dbo.DimProduct AS p
         WHERE  p.ProductAlternateKey = ref.ProductAlternateKey
           AND  p.StartDate <= f.OrderDate
         ORDER  BY p.StartDate DESC
         LIMIT  1
       ) AS v ON TRUE;
```

Théorie : `Theorie_09_SQL.md`, section 6.

E5. Le défaut visible est le tri : `ORDER BY pl.modifieddate DESC` ne trie pas sur la colonne qui porte la comparaison. La date de modification est une date technique, celle où la ligne a été touchée dans la base, et rien ne garantit qu'elle suive l'ordre des périodes de validité. Sur `Adventureworks`, elle ne le suit pas : pour 59 produits sur 293, ce tri désigne une autre version que le tri par `startdate`, donc un autre prix. Il faut écrire `ORDER BY pl.startdate DESC`.

Le défaut invisible est l'absence de `LEFT` : une ligne de commande dont le produit n'a aucun prix antérieur disparaîtrait du résultat. Sur cette base, toutes les lignes trouvent une version, donc le défaut ne se manifeste pas, et c'est précisément ce qui le rend dangereux. Il attend un jeu de données un peu différent.

Théorie : `Theorie_09_SQL.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

E6. Une erreur, et un manque qui n'en est pas une.

L'erreur est dans le `LAG` de la CTE, qui n'a pas de `PARTITION BY customerid` : il compare chaque commande à la précédente toutes commandes confondues, et le drapeau ne signale plus les ruptures d'un client mais les trous du flux global. Comme les commandes de tous les clients se succèdent de près, presque aucune rupture ne sort, et la plupart des clients se retrouvent avec une seule session. C'est le symptôme décrit.

Le second point, celui de la somme cumulée, se lit souvent comme une erreur et n'en est pas une. La fenêtre n'a pas de cadre explicite, donc elle hérite de `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. Comme `orderdate` est à minuit pour les 31 465 commandes de cette base, deux commandes d'un client le même jour portent la même valeur d'ordre : elles sont pairs, `RANGE` les additionne d'un coup, et elles reçoivent le même numéro de session. C'est exactement ce qu'on veut d'une sessionisation : deux commandes du même jour appartiennent à la même session. Remplacer par `ROWS` leur donnerait deux numéros différents, ce qui serait le vrai bogue. Ici, 59 couples client et jour sont concernés.

Ce qui manque vraiment est ailleurs, et c'est un défaut de déterminisme : l'ordre `ORDER BY orderdate` ne départage pas les commandes du même jour, donc deux exécutions peuvent les présenter dans deux ordres différents. Le remède est un départage explicite, `ORDER BY orderdate, salesorderid`, dans les deux fenêtres. Il ne change pas les numéros de session, il rend la sortie reproductible.

```sql
WITH marque AS (
    SELECT customerid, orderdate,
           CASE WHEN orderdate - LAG(orderdate) OVER (PARTITION BY customerid
                                                      ORDER BY orderdate, salesorderid)
                     > INTERVAL '120 days'
                THEN 1 ELSE 0 END AS rupture
    FROM   sales.salesorderheader
)
SELECT customerid, orderdate,
       SUM(rupture) OVER (PARTITION BY customerid ORDER BY orderdate) AS session
FROM   marque;
```

Théorie : `Theorie_09_SQL.md`, section 7.

E7. Le diviseur fixe. Le 8 mars, la journée de Montréal n'a que 23 heures : la somme ne porte que 23 relevés, et la diviser par 24 sous-estime la puissance moyenne d'un vingt-quatrième. Le 1er novembre, elle en a 25, et la division par 24 la surestime d'autant. La courbe creuse au printemps et bondit à l'automne, sans qu'aucune machine n'ait changé. Le correctif divise par le nombre d'heures réellement présentes, qui est aussi le seul diviseur juste un jour où il manque un relevé :

```sql
SELECT CAST(instant AT TIME ZONE 'America/Toronto' AS DATE) AS jour,
       SUM(kwh) / COUNT(*)                                  AS kw_moyens
FROM   releves_horaires
GROUP  BY 1;
```

Si la série peut avoir des trous, `COUNT(*)` compte les heures mesurées, pas les heures écoulées : la question devient alors celle de l'axe complet, à la section 2.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_09_SQL.md`, sections 1 et 2.

Questions à développer

A1. Une bonne réponse défend la règle par ce qu'elle rend inutile. L'intervalle fermé demande deux informations, un début et une fin, et il suppose en plus que les intervalles pavent l'axe sans trou ni recouvrement. Cela fait trois hypothèses sur la qualité d'une donnée que personne n'a vérifiée. La date de début seule n'en demande aucune : la version en vigueur est la plus récente qui a commencé, et cette définition se suffit à elle-même. Elle survit donc à une fin absente, à une fin décalée, et à une fin franchement fausse comme celle de `dbo.DimProduct`.

Ce que la règle coûte doit aussi être dit, et c'est ce qui distingue une bonne réponse. L'écriture par date de début rattache toujours quelque chose dès qu'une version antérieure existe, y compris lorsqu'une période de validité s'est réellement terminée sans qu'une suivante ait commencé. Un produit retiré du catalogue en 2023 se verra attribuer son dernier prix connu pour une commande de 2025, alors que l'intervalle fermé aurait dit « rien ». Sur une valorisation, ce report est souvent le comportement voulu, puisqu'on veut la dernière valeur connue. Sur un contrôle de conformité, il ne l'est pas, et il faut alors ajouter explicitement la borne de fin, en la contrôlant d'abord.

Est également acceptable une réponse qui propose de calculer la fin plutôt que de la lire, par un `LEAD` sur la date de début de la version suivante : c'est une bonne parade, elle rend les intervalles cohérents par construction, et elle mérite d'être connue.

Théorie : `Theorie_09_SQL.md`, section 6.

A2. Les trois lectures sont les suivantes.

Les dates de la dimension sont fausses, et `StartDate` désigne en réalité un millésime de catalogue plutôt qu'une date de validité. C'est la lecture que l'incohérence massive des dates de fin rend la plus crédible : une table dont 200 fins sur 200 sont fausses n'inspire pas confiance sur ses débuts non plus.

L'alimentation du fait est fausse, et elle a rattaché la version courante au lieu de la version du jour. C'est une erreur classique d'un chargement écrit à la hâte, et elle se teste : il suffit de regarder si les désaccords pointent systématiquement vers une ligne dont le statut est `Current`.

Les deux mesures ne décrivent pas la même chose. La clé du fait pourrait dire « la version connue au moment du chargement », ce qui est une information légitime et différente de « la version valide au moment de la vente ».

La mesure qui tranche entre les deux premières est la requête de la partie 3.3 du laboratoire OLAP. Si les désaccords pointent presque tous vers la version courante, la deuxième lecture est la bonne. S'ils se répartissent, la première est plus probable. Une bonne réponse ajoute ce qu'il faut faire faute de pouvoir trancher : on documente l'écart, on choisit une des deux définitions, on l'écrit dans la documentation du rapport, et on ne laisse surtout pas deux rapports du même service employer chacun la sienne.

Théorie : `Theorie_09_SQL.md`, sections 6 et 10.

A3. Le monde transactionnel place l'historique dans une table dédiée parce que sa table principale doit rester petite et rapide en écriture : `production.product` porte l'état courant, une ligne par produit, et c'est cette table que des milliers de transactions lisent chaque minute. Y garder toutes les versions la ferait grossir

© 2026 iAlexMG. Tous droits réservés.

sans profit pour personne. La table d'historique est alimentée par l'application au moment du changement, et le moteur peut y poser des contraintes, ce qu'il fait ici puisque la définition exige que la fin suive le début.

L'entrepôt replie l'historique dans la dimension parce que son problème est inverse : il doit pouvoir rattacher une ligne de fait vieille de trois ans au contexte qui avait cours à ce moment-là, et une jointure supplémentaire sur une table d'historique coûterait cher à chaque requête. La dimension à évolution lente résout cela en donnant une clé technique à chaque version, si bien que le fait pointe directement vers le bon contexte et qu'aucune logique temporelle n'est nécessaire à la lecture.

Les erreurs ne se logent donc pas au même endroit. Du côté transactionnel, elles viennent de l'application qui alimente l'historique, et une contrainte peut en attraper une partie. Du côté de l'entrepôt, elles viennent du traitement d'alimentation qui décide quand créer une version et quelle clé attribuer au fait ; ce traitement n'est surveillé par aucune contrainte, et les deux défauts observés dans ce module en viennent tous les deux.

Théorie : `Theorie_09_SQL.md`, sections 6 et 10.

A4. Ce que les trois fautes ont en commun est le mode de défaillance, pas la cause. Aucune ne produit d'erreur, et toutes les trois produisent un résultat qui a la bonne forme : zéro ligne, une valeur `NULL`, une liste triée. Le SQL a fait exactement ce qu'on lui a demandé, et c'est la demande qui était fausse. Une deuxième chose les rapproche : dans les trois cas, le comportement fautif dépend de la donnée et non du code, si bien que la même requête peut être juste sur le jeu d'essai et fausse en production.

La pratique d'équipe qui les attraperait toutes les trois est le test de dénombrement. Après toute jointure ou tout filtre susceptible de perdre des lignes, on compte avant et après, et on fait échouer le traitement si l'écart dépasse un seuil déclaré. Cette pratique attrape le `NOT IN` qui vide un résultat, la valeur `JSON` comparée sans la flèche double qui renvoie zéro ligne, et l'as-of join qui laisse des lignes sans correspondance, parce que ces trois fautes se manifestent toutes par un compte qui change.

Une réponse qui propose plutôt la revue de code, ou l'interdiction des écritures dangereuses par une règle de style, est recevable si elle reconnaît la limite : ces deux pratiques dépendent de l'attention humaine, et les trois fautes du module sont précisément celles qui ne réveillent l'attention de personne. Le test de dénombrement, lui, ne se fatigue pas.

Théorie : `Theorie_09_SQL.md`, sections 1 et 6.

Mises en situation

M1. Réponse défendable.

C'est un as-of join, avec la table des ordres à gauche et la table des cours à droite, apparié sur le symbole et sur l'horodatage. La sous-requête latérale prend les cours du même symbole dont l'horodatage est antérieur ou égal à celui de l'ordre, les trie par horodatage décroissant, et n'en garde qu'un. Le `LEFT` est obligatoire, la comparaison porte sur la colonne qui sert aussi au tri, et la clause d'égalité porte sur le symbole seul.

© 2026 iAlexMG. Tous droits réservés.

Les ordres sans cours antérieur doivent rester visibles, et c'est le point que l'énoncé cherche. Un ordre exécuté avant le premier cours connu signale soit un trou dans l'alimentation des cours, soit un instrument nouvellement traité dont l'historique n'a pas été chargé. Les faire disparaître par une jointure interne masquerait exactement l'anomalie que la conformité cherche à voir, et le rapport serait complet en apparence.

Si la conformité demandait le cours suivant plutôt que le précédent, il suffirait de retourner les deux signes : `>=` dans la comparaison et `ASC` dans le tri. C'est le même outil. Une bonne réponse ajoute que les deux ensemble encadrent l'exécution, ce qui est la façon usuelle de mesurer la qualité d'un prix d'exécution, et qu'un décalage de quelques millisecondes suffit à faire basculer d'un côté ou de l'autre, donc que la convention sur l'égalité doit être écrite.

Théorie : `Theorie_09_SQL.md`, section 6.

M2. La construction attendue tient en trois étapes.

D'abord l'axe : on génère la liste des jours ouvrables des trois dernières années, puis on la croise avec la liste des cinq échéances. Le produit cartésien est ici légitime et voulu, puisqu'on veut une ligne par jour et par échéance, y compris les jours où rien n'a bougé. C'est le geste du Cours 05, appliqué à deux dimensions.

Ensuite l'as-of join : pour chaque case de cette grille, on va chercher dans l'historique des révisions la dernière révision de cette échéance dont la date d'entrée en vigueur est antérieure ou égale au jour. C'est le geste du Cours 09.

Enfin la mise en forme, qui n'est qu'un `PIVOT` du Cours 06 si l'on veut cinq colonnes plutôt que cinq lignes.

La question ne peut pas se résoudre par une jointure d'égalité parce qu'il n'existe pas de ligne de révision pour chaque jour : une révision vaut jusqu'à la suivante, et la plupart des jours n'ont aucune ligne à leur date. Une jointure d'égalité ne renverrait donc que les jours de révision, soit quelques dizaines de lignes au lieu de plusieurs milliers, et le graphique montrerait une courbe en pointillés au lieu d'un escalier.

Sans la première étape, le résultat serait exactement ce cas : on n'obtiendrait que les jours où une révision a eu lieu. L'axe est ce qui transforme un journal de changements en série continue.

Le début de la plage se traite explicitement, et une bonne réponse le dit. Avant la première révision connue, l'as-of join ne trouve rien et renvoie `NULL`. Trois traitements se défendent : laisser `NULL`, ce qui est honnête ; reculer le chargement de l'historique pour couvrir le début de la plage, ce qui est la vraie solution ; ou reporter la première valeur connue vers l'arrière, ce qui est une invention et doit alors être signalé dans le tableau.

Théorie : `Theorie_09_SQL.md`, sections 2 et 6.

M3. Réponse défendable.

Le piège que l'énoncé ne nomme pas est le grain irrégulier. Une fenêtre de vingt-quatre heures écrite en cadre de lignes suppose 144 mesures par jour ; sur une turbine dont la télémétrie s'est interrompue trois heures, le cadre de 144 lignes remonte à vingt-sept heures en arrière, et sur une turbine sans interruption il remonte à vingt-quatre. Les écarts types des turbines ne portent donc pas sur la même durée, et la comparaison demandée par l'exploitant, qui est le but de tout l'exercice, n'a plus de sens. Le cadre en lignes fabrique une différence entre turbines qui ne mesure que la qualité de leur télémétrie.

Il faut donc le cadre en durée, ou bien un axe complet à dix minutes suivi d'un cadre en lignes, ce qui revient au même et se porte mieux d'un moteur à l'autre.

© 2026 iAlexMG. Tous droits réservés.

Le sort des interruptions se tranche par la nature de la grandeur, et c'est le cœur de la réponse. Une absence de télémétrie n'est pas une production nulle : mettre zéro invente une chute de production et gonfle l'écart type, ce qui déclencherait des alertes sur les turbines les moins bien instrumentées. Laisser la valeur manquante, et donc l'exclure de la fenêtre, est la seule option honnête : l'écart type se calcule sur les mesures existantes, et il reste comparable tant qu'il en reste assez.

La dernière partie de l'énoncé demande l'information à ajouter, et c'est elle qui distingue une très bonne réponse. Il faut publier, à côté de l'écart type, le nombre de mesures réellement présentes dans la fenêtre, ou la part de la fenêtre effectivement couverte. Un écart type calculé sur douze mesures au lieu de 144 n'est pas faux, il est peu fiable, et seule cette colonne permet au lecteur de le savoir. On peut aller plus loin en masquant la valeur sous un seuil de couverture déclaré, par exemple soixante-dix pour cent, ce qui transforme une décision de lecture en règle explicite.

Théorie : `Theorie_09_SQL.md`, sections 4 et 5.

© 2026 iAlexMG. Tous droits réservés.